Lab Task 1:

Translate the linear search algorithm into a program which either find the location LOC where ITEM appears in ARRAY or return LOC=0

Code:

```
#include <iostream>
using namespace std;

int main()
{
	const int arrSize = 10;
	int arr[arrSize] = { 21, 99, 150, 212, 320, 420, 510, 603, 834 , 911 };
	int item = 603;
	int loc = 0;

	for (int i = 0; i < arrSize; i++) {
		if (arr[i] == item) {
			loc = i + 1;
			break;
		}
	}

	if (loc != 0) {
		cout << "Item " << item << " found at location " << loc << endl;
	}
	else {
		cout << "Item not found in the array." << endl;
	}
}
```

Output:

```
Item 603 found at location 8
```

Lab Task 2:

Translate binary search and insertion algorithm into a program which finds either the location LOC where ITEM appears in ARRAY or the location LOC where ITEM should be inserted into ARRAY. (For object 3, 4, and 5 take an array of 10 elements given below)

Code:

```
#include <iostream>
using namespace std;

int main()
{
	const int ArrSize = 10;
	int arr[ArrSize] = { 3, 5, 8, 12, 15, 18, 20, 25, 30, 35 };
	int item;

	cout << "Enter item to search: ";
	cin >> item;

	int beg = 0;
	int end = ArrSize - 1;
	int loc = -1;

	while (beg <= end)
	{
		int mid = (beg + end) / 2;
		if (arr[mid] == item) {
			loc = mid + 1;
			break;
		}
		else if (item < arr[mid]) {
			end = mid - 1;
		}
		else {
			beg = mid + 1;
		}
	}

	if (loc != -1) {
		cout << "Item " << item << " found at location " << loc << endl;
	}
	else {
		cout << "Item not found in the array." << endl;
		cout << "Item should be inserted at location " << beg + 1 << endl;
	}
}
```

Output:

```
Enter item to search: 8
Item 8 found at location 3
```

Lab Task 3:

Write a program which uses Binary Search to search the elements52, and 33 and print it.

Code:

```
#include <iostream>
using namespace std;

int main() {
    const int n = 10;
    int arr[n] = { 11, 22, 33, 36, 45, 52, 57, 60, 64, 78 };

    int items[2] = { 52, 33 };

    for (int k = 0; k < 2; k++) {
        int item = items[k];
        int beg = 0;
        int end = n - 1;
        int loc = -1;

        while (beg <= end) {
            int mid = (beg + end) / 2;
            if (arr[mid] == item) {
                loc = mid;
                break;
            }
            else if (item < arr[mid]) {
                end = mid - 1;
            }
            else {
                beg = mid + 1;
            }
        }

        if (loc != -1) {
            cout << item << " found at index " << loc << endl;
        }
        else {
            cout << item << " not found" << endl;
        }
    }
}
```

Output:

```
52 found at index 5
33 found at index 2
```

Lab Task 4:

Write a program which uses Binary Search to search the elements 45 and 78 and delete it. NUMBERS= [11,22,33,36,45,52,57,60,64,78]

Code:

```
#include <iostream>
using namespace std;

int main()
{
    const int arrSIze = 10;
    int arr[arrSIze] = { 11, 22, 33, 36, 45, 52, 57, 60, 64, 78 };
    int n = arrSIze;
    int items[2] = { 45, 78 };

    for (int k = 0; k < 2; k++)
    {
        int item = items[k];
        int beg = 0, end = n - 1;
        int loc = -1;
        while (beg <= end)
        {
            int mid = (beg + end) / 2;

            if (arr[mid] == item)
            {
                loc = mid;
                break;
            }
            else if (item < arr[mid])
            {
                end = mid - 1;
            }
            else
            {
                beg = mid + 1;
            }
        }
        if (loc != -1)
        {
            for (int i = loc; i < n - 1; i++)
            {
                arr[i] = arr[i + 1];
            }
            n--;
        }
    }

    cout << "Updated Array: ";
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}
```

Output:

```
Updated Array: 11 22 33 36 52 57 60 64
```

Lab Task 1:

Summarize the time and space complexity of algorithms and list the key differences in a table.

SUMMARY:

Time Complexity: Time complexity reveals how the quantity of operations an algorithm performs expands as the input volume (n) increases. It illustrates the scaling behavior of runtime with growing data size, instead of tracking real-world seconds.

Space Complexity: Space complexity denotes the volume of memory an algorithm utilizes in proportion to the input scale (n). This covers storage for input data, output results, auxiliary variables, and stack space from recursive calls or function invocations.

DIFFERENCES BETWEEN SPACE AND TIME COMPLEXITY:

Aspect	Time Complexity	Space Complexity
Definition	Indicates how the execution duration of an algorithm expands as the input volume increases.	Shows the quantity of memory an algorithm demands as the input scale grows.
Focus	Runtime performance or processing speed.	Memory consumption or storage needs.
Measurement	Number of fundamental operations or computational steps performed.	Volume of memory units (variables, bytes, or auxiliary space) utilized.
Main Concern	CPU efficiency or processor workload.	Memory (RAM or storage) availability.
Optimization Goal	Minimize the duration required for completion.	Decrease the overall memory footprint.
Types	Best case, average case, worst case (e.g., O(1), O(log n), O(n), O(n^2))	Usually denoted in similar fashion (e.g., O(1), O(n), O(n^2)).

Lab Task 2:

Determine, for the typical algorithms that you use to perform calculations by hand, the running time to do the following:
a. Add two N-digit integers.
b. Multiply two N-digit integers.
c. Divide two N-digit integers.

a. Addition of Two N-Digit Numbers:

Process: Sum corresponding digits from the least significant position toward the most significant, propagating any carry to the following position.
Explanation: A single addition is needed per digit pair; hence the total effort rises linearly with the count of digits.
Time Complexity: O(N)

b. Multiplication of Two N-Digit Numbers:

Process: Compute the product of every digit from the first number with each digit of the second, then combine all intermediate products (similar to manual long multiplication).
Explanation: Each digit in one operand pairs with every digit in the other, producing approximately N×N individual multiplications.
Time Complexity: $O(N^2)$

c. Division of Two N-Digit Numbers:

Process: Apply classic long division by repeatedly subtracting scaled versions of the divisor from the current remainder and appending the next digit.
Explanation: Every iteration performs comparisons and subtractions that may span up to N digits, yielding around N^2 elementary operations overall.
Time Complexity: $O(N^2)$

SUMMARY TABLE:

Operation	Running Time (Big-O)
Addition of two N-digit integers	O(n)
Multiplication of two N-digit integer	O(n2)
Division of two N-digit integers	O(n2)

Lab Task 3:

Describe all cases of algorithm analysis (BEST, WORST & AVERAGE CASES).
For a sequential search of an array a[0..n] with n elements write the comparison of these cases.

Cases of Algorithm Analysis:

Algorithm analysis is the technique used to evaluate how efficient an algorithm performs. It is typically expressed through three scenarios:

- **Best Case:** The situation in which the algorithm requires the least amount of time to complete.
- **Worst Case:** The scenario where the algorithm demands the maximum possible time to finish execution. ·
- **Average Case:** The anticipated runtime of the algorithm when operating on ordinary or randomly distributed inputs.

SEQUENTIAL SEARCH:

Case	Condition	Number of Comparisons	Time Complexity	Explanation
Best Case	The target element K is positioned at the very start of the array.	1	O(1)	The search terminates instantly after the first check.
Worst Case	The target element K is located at the end of the array or is absent entirely.	N	O(n)	Every element in the array needs to be examined before reaching a conclusion.
Average Case	The target element K can appear in any random position with equal likelihood.	(n+1)/2	O(n)	On average, roughly half of the elements are inspected before locating K.

Lab Task 1:

Write the program for the Bubble Sort algorithm and explain its time complexity, advantages, and disadvantages.

Code:

```
#include <iostream>
using namespace std;

int main()
{
    int arr[10];

    for (int i = 0; i < 10; i++) {
        cout << "arr[" << i << "]= ";
        cin >> arr[i];
    }

    for (int i = 0; i < 10 - 1; i++) {
        for (int j = 0; j < 10 - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    cout << "\nSorted Array: ";
    for (int i = 0; i < 10; i++) {
        cout << arr[i] << " ";
    }
}
```

Output:

```
arr[0]= 55
arr[1]= 54
arr[2]= 685
arr[3]= 2
arr[4]= 1
arr[5]= 322
arr[6]= 2
arr[7]= 6
arr[8]= 10
arr[9]= 655

Sorted Array: 1 2 2 6 10 54 55 322 655 685
```

Lab Task 2:

Write the program for the Selection Sort algorithm and explain its time complexity, advantages, and disadvantages.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[10];

    for (int i = 0; i < 10; i++) {
        cout << "arr[" << i << "]= ";
        cin >> arr[i];
    }

    for (int i = 0; i < 10 - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < 10; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        int temp = arr[i];
        arr[i] = arr[minIndex];
        arr[minIndex] = temp;
    }

    cout << "\nSorted Array: ";
    for (int i = 0; i < 10; i++) {
        cout << arr[i] << " ";
    }
}
```

Output:

```
arr[0]= 84
arr[1]= 35
arr[2]= 15
arr[3]= 65
arr[4]= 356
arr[5]= 156
arr[6]= 22
arr[7]= 753
arr[8]= 553
arr[9]= 1551

Sorted Array: 15 22 35 65 84 156 356 553 753 1551
```

Lab Task 3:

Write the program for the Insertion Sort algorithm and explain its time complexity, advantages, and disadvantages.

Code:

```
#include <iostream>
using namespace std;

int main() {
    int arr[10];

    for (int i = 0; i < 10; i++) {
        cout << "arr[" << i << "]= ";
        cin >> arr[i];
    }

    for (int i = 1; i < 10; i++) {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }

    cout << "\nSorted Array: ";
    for (int i = 0; i < 10; i++) {
        cout << arr[i] << " ";
    }
}
```

Output:

```
arr[0]= 54
arr[1]= 684
arr[2]= 57
arr[3]= 6582
arr[4]= 15
arr[5]= 5
arr[6]= 8
arr[7]= 54
arr[8]= 65
arr[9]= 6

Sorted Array: 5 6 8 15 54 54 57 65 684 6582
```